

INSTITUTO FEDERAL DA PARAÍBA – IFPB

DISCIPLINA: Sistemas Embarcados

PROFESSOR: Alexandre Sales Vasconcelos

EQUIPE: Bráulio Matheus Brito Barbosa

CURSO: Engenharia de Computação

Entrega: Relatório da Atividade 09

## **Sistema Multitarefa com FreeRTOS, MPU6050 e Controle PWM (ESP32-S3)**

### **Descrição do Projeto**

Projeto desenvolvido utilizando o ESP32-S3 DevKitC-1 no simulador Wokwi, com o objetivo de implementar uma arquitetura multitarefa baseada em FreeRTOS. O sistema integra leitura analógica, controle PWM, comunicação I2C e mecanismos de sincronização entre tarefas, permitindo o monitoramento e controle de diferentes periféricos em tempo real.

### **Link da Simulação**

Projeto no Wokwi:

<https://wokwi.com/projects/463909116911517697>

### **Objetivos**

Implementar um sistema embarcado capaz de:

- Ler continuamente um potenciômetro utilizando o ADC do ESP32-S3;
- Controlar o brilho de um LED através de PWM;
- Alternar entre os modos LIVE e HOLD por meio de um botão;
- Realizar a leitura de aceleração utilizando o sensor MPU6050 via I2C;
- Converter os valores brutos do acelerômetro para unidades de aceleração gravitacional (g);
- Utilizar mecanismos de comunicação e sincronização entre tarefas do FreeRTOS;
- Exibir informações do sistema em tempo real através da porta serial.

### **Componentes Utilizados**

- ESP32-S3 DevKitC-1;
- 1 LED vermelho;
- 1 resistor de  $220\ \Omega$ ;
- 1 potenciômetro de  $10\ \text{k}\Omega$ ;
- 1 botão (pushbutton);
- 1 resistor de  $10\ \text{k}\Omega$  (pull-down);
- 1 sensor MPU6050;
- Protoboard;
- Jumpers.

## **Ligações do Circuito**

### **LED PWM**

- GPIO 4  $\rightarrow$  Resistor  $220\ \Omega \rightarrow$  Ânodo do LED;
- Cátodo do LED  $\rightarrow$  GND.

### **Potenciômetro**

- VCC  $\rightarrow$  3,3 V;
- GND  $\rightarrow$  GND;
- Terminal central (SIG)  $\rightarrow$  GPIO 1.

### **Botão**

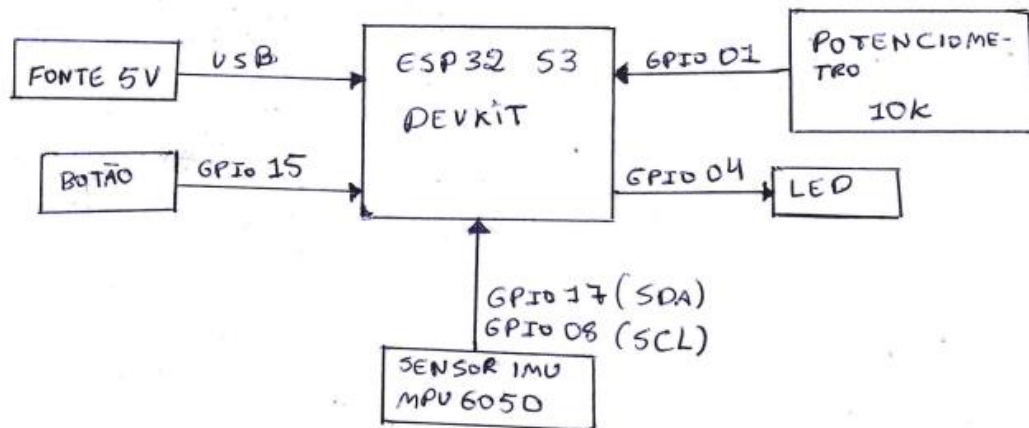
- Um terminal  $\rightarrow$  3,3 V;
- Outro terminal  $\rightarrow$  GPIO 15;
- Resistor de  $10\ \text{k}\Omega$  entre GPIO 15 e GND.

### **Sensor MPU6050**

- VCC  $\rightarrow$  3,3 V;
- GND  $\rightarrow$  GND;
- SDA  $\rightarrow$  GPIO 8;
- SCL  $\rightarrow$  GPIO 9.

### Diagrama de Blocos

DIAGRAMA DE BLOCOS - ATIVIDADE 09



### Diagrama Esquemático

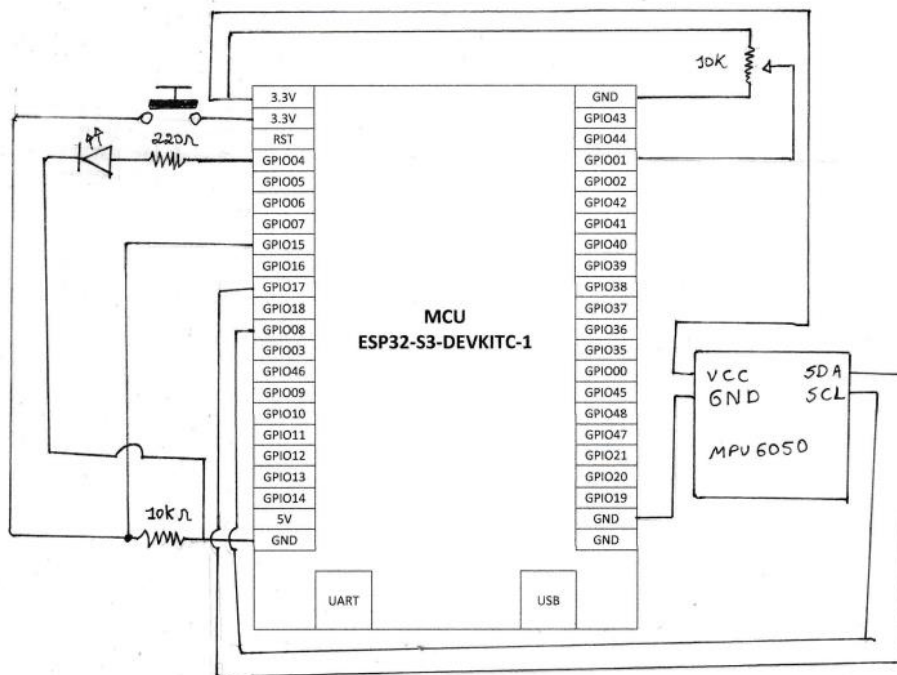


DIAGRAMA ESQUEMATICO ATV 09-RT05 (PRÁTICA)

## **Arquitetura Multitarefa**

O firmware foi dividido em cinco tarefas independentes executadas pelo FreeRTOS.

### **Tarefa Potenciômetro**

Responsável por:

- Ler o valor analógico do potenciômetro;
- Converter a leitura para a faixa utilizada pelo PWM;
- Enviar os valores para uma fila compartilhada.

### **Tarefa LED**

Responsável por:

- Receber os valores provenientes da fila;
- Atualizar o duty cycle do PWM;
- Ajustar o brilho do LED.

### **Tarefa Botão**

Responsável por:

- Monitorar o estado do botão;
- Detectar acionamentos;
- Liberar um semáforo binário para alternar o estado do sistema.

### **Tarefa IMU**

Responsável por:

- Ler os registradores do MPU6050 via I2C;
- Obter os valores dos eixos X, Y e Z;
- Converter os valores para aceleração em g;
- Atualizar os dados compartilhados.

### **Tarefa Console**

Responsável por:

- Exibir os dados do sistema no terminal serial;
- Mostrar o estado atual do sistema;
- Apresentar as leituras do potenciômetro, LED e MPU6050.

## **Modos de Operação**

### **Modo LIVE**

Características:

- O LED acompanha continuamente as variações do potenciômetro;

- Toda alteração realizada no potenciômetro é refletida imediatamente no brilho do LED.

### **Modo HOLD**

Características:

- O brilho do LED permanece congelado no último valor recebido;
- As leituras do potenciômetro continuam ocorrendo normalmente;
- As demais tarefas permanecem em execução;
- Um novo acionamento do botão retorna o sistema ao modo LIVE.

## **Mecanismos de Comunicação e Sincronização**

### **Queue (Fila)**

Comunicação:

- Tarefa Potenciômetro → Tarefa LED

Função:

- Transferir os valores do ADC para a tarefa responsável pelo PWM;
- Garantir que os dados sejam recebidos de forma organizada e sequencial.

### **Semáforo Binário**

Comunicação:

- Tarefa Botão → Controle de Estado

Função:

- Sinalizar o acionamento do botão;
- Alternar entre os modos LIVE e HOLD.

### **Mutex**

Comunicação:

- Tarefa IMU ↔ Tarefa Console

Função:

- Proteger o acesso aos dados compartilhados do acelerômetro;
- Evitar leituras inconsistentes durante atualizações simultâneas.

## **Configurações dos Periféricos**

### **ADC**

- Resolução: 12 bits;
- Faixa de leitura: 0 a 4095;
- Tensão de referência aproximada: 0 a 3,3 V.

## **PWM**

- Frequência: 5 kHz;
- Resolução: 13 bits;
- Duty Cycle máximo: 8191.

## **I2C**

- Frequência: 100 kHz;
- SDA: GPIO 8;
- SCL: GPIO 9.

## **MPU6050**

- Endereço I2C: 0x68;
- Escala do acelerômetro:  $\pm 2$  g;
- Sensibilidade: 16384 LSB/g.

## **Exemplo de Saída Serial**

```
=====
STATUS: [LIVE] | POT: 2048 (1650 mV) | LED: 50%
IMU ACCEL (g): X: 0.02 | Y: -0.10 | Z: 0.98
=====
=====
STATUS: [HOLD] | POT: 3500 (2820 mV) | LED: 85%
IMU ACCEL (g): X: 0.15 | Y: -0.02 | Z: 0.97
=====
```

## **Tecnologias Utilizadas**

- Linguagem C;
- ESP-IDF;
- FreeRTOS;
- ADC One-Shot Driver;
- LEDC PWM Driver;
- Driver I2C;
- Sensor MPU6050;
- Wokwi Simulator.

## **Conceitos Aplicados**

- Sistemas Operacionais de Tempo Real (RTOS);
- Multitarefa;

- Escalonamento de tarefas;
- Filas (Queues);
- Semáforos Binários;
- Mutexes;
- Comunicação entre tarefas;
- Conversão Analógico-Digital (ADC);
- Modulação por Largura de Pulso (PWM);
- Comunicação I2C;
- Leitura de sensores inerciais;
- Programação embarcada com ESP32-S3.

### **Possíveis Melhorias**

- Implementação de interrupções para o botão;
- Leitura do giroscópio do MPU6050;
- Aplicação de filtros digitais para suavização das leituras;
- Integração com display OLED;
- Armazenamento de dados em cartão SD;
- Comunicação Wi-Fi utilizando MQTT;
- Dashboard web para monitoramento remoto;
- Registro histórico das medições;
- Ajuste dinâmico das prioridades das tarefas.

### **Conclusão**

O projeto demonstrou a aplicação prática dos principais recursos do FreeRTOS em um sistema embarcado utilizando o ESP32-S3. A utilização de tarefas independentes, filas, semáforos e mutexes permitiu a implementação de uma arquitetura organizada e eficiente, possibilitando a integração simultânea de múltiplos periféricos e garantindo a comunicação segura entre as diferentes partes do sistema. Além disso, o projeto serviu como exercício prático de conceitos fundamentais de sistemas operacionais de tempo real, comunicação I2C, controle PWM e aquisição de dados analógicos.